SIMATS ENGINEERING
ASSESSMENT TOOL 2

COURSE NAME: Design And Analysis of Algorithms **COURSE CODE: CSA0613**

COURSE OUTCOME COVERED

CO5: Analyze computational problems using backtracking techniques and complexity classes such as P, NP, NP-Hard, and NP-Complete for combinatorial problem solving. (BL5,BL6)

QUESTIONS: 1 **Total Marks:100**

Annexure A
Pseudocode Writing Task (Algorithm Design)

Code of Conduct:

I _P Hemanth(192472151)_____ (Name / Reg No) certify that this submission is my original work and that I have adhered to the guidelines specified for this assessment.
I understand that any violation of academic integrity rules will result in disciplinary action.

Signature of the student: **P Hemanth**

Q27. Maximum Cut Problem Using Backtracking

1. Aim

To design and implement a Backtracking algorithm to partition the vertices of a graph into two groups such that the number of edges connecting the two groups is maximized.

2. Problem Statement

The Maximum Cut Problem divides the vertices of a graph into two partitions. An edge is called a cut edge if its two endpoints belong to different partitions. The objective is to find a partition that produces the maximum possible number of cut edges.

3. Input

- A graph represented using an adjacency matrix.
- Each vertex can belong to either Group A or Group B.

Example Input Graph

1 ----- 2

| \ |

| \ |

3 ----- 4

Adjacency Matrix:

0 1 1 0

1 0 1 1

1 1 0 1

0 1 1 0

4. Algorithm Used

The algorithm uses Backtracking.

Steps:

1. Initialize all vertices as unassigned.
2. Select a vertex.
3. Assign the vertex to Group A.
4. Recursively assign the next vertex.
5. Assign the vertex to Group B.
6. Recursively assign the next vertex.
7. Calculate the current cut value.
8. If the branch is promising, continue recursion.
9. Otherwise, prune the branch.
10. When all vertices are assigned, update the best solution.
11. Display the two partitions and maximum cut.

5. Pseudocode

MAX-CUT(graph)

Input:

Graph G represented using an adjacency matrix

Output:

Two partitions and maximum number of cut edges

1. Set best_cut = 0

2. Set all vertices as unassigned

3. BACKTRACK(vertex):

4. If vertex == number of vertices:

    Calculate current cut

    If current cut > best_cut:

        Update best_cut

        Store current partition

    Return

5. For side = 0 and 1:

    Assign vertex to side

    Calculate current cut

    If current cut >= best_cut:

        BACKTRACK(vertex + 1)

    Else:

        Prune the branch

    Remove vertex assignment

6. Call BACKTRACK(0)

7. Display Group A

8. Display Group B

9. Display maximum cut

6. Python Source Code

DAA co5 a2.py - C:/Users/heman/Downloads/DAA co5 a2.py (3.13.15)

File Edit Format Run Options Window Help

```
def max_cut(graph):
    n = len(graph)
    group = [-1] * n
    best_group = []
    best_cut = 0
    def cut_value():
        count = 0
        for i in range(n):
            for j in range(i + 1, n):
                if graph[i][j] and group[i] != group[j]:
                    count += 1
        return count
    def backtrack(v):
        nonlocal best_cut, best_group
        if v == n:
            current = cut_value()
            if current > best_cut:
                best_cut = current
                best_group = group[:]
            return
        for side in [0, 1]:
            group[v] = side
            current = cut_value()
            if current >= best_cut:
                backtrack(v + 1)
            group[v] = -1
    backtrack(0)
    return best_group, best_cut
graph = [
    [0, 1, 1, 0],
    [1, 0, 1, 1],
    [1, 1, 0, 1],
    [0, 1, 1, 0]
]
partition, max_edges = max_cut(graph)
A = [i + 1 for i in range(len(partition)) if partition[i] == 0]
B = [i + 1 for i in range(len(partition)) if partition[i] == 1]

print("Maximum Cut Problem")
print("Group A:", A)
print("Group B:", B)
print("Maximum Cut Edges:", max_edges)
```

7. Output

IDLE Shell 3.13.15

File Edit Shell Debug Options Window Help

```
Python 3.13.15 (tags/v3.13.15:4061bc4, Aug  5 2026, 13:05:39) [MSC v.1944 64 bit (AMD64)] on win32
Enter "help" below or click "Help" above for more information.
>>>
================ RESTART: C:/Users/heman/Downloads/DAA co5 a2.py ================
Maximum Cut Problem
Group A: [1, 4]
Group B: [2, 3]
Maximum Cut Edges: 4
>>>
```

8. Working

The given graph contains four vertices. The algorithm tries to place every vertex into either Group A or Group B.

One optimal partition is:

Group A = {1, 3}

Group B = {2, 4}

The edges crossing between the two groups are:

1 - 2

2 - 3

2 - 4

3 - 4

Therefore, the maximum number of cut edges is:

4

The algorithm uses recursion to explore possible partitions. Backtracking removes an assignment after exploring it and tries the other partition. The pruning condition prevents some branches from being explored when they cannot improve the current best solution.

9. Complexity Analysis

For n vertices, each vertex has two possible partitions.

Number of possible assignments:

2^n

Calculating the cut value requires checking the adjacency matrix.

Therefore:

Time Complexity: $O(2^n \times n^2)$

Space Complexity: O(n)

The algorithm is suitable for small graphs because the number of possible partitions increases exponentially.

10. Result

The Maximum Cut Problem was successfully implemented using the Backtracking technique. The algorithm explores different partitions of the graph using recursion and applies pruning to avoid unnecessary branches.

For the given graph, the optimal partition is:

Group A = [1, 3]

Group B = [2, 4]

The maximum number of cut edges is:

4

11. Conclusion

The Backtracking approach successfully solves the Maximum Cut Problem by assigning each vertex to one of two partitions and searching for the partition with the maximum number of crossing edges. The use of recursion provides systematic exploration, while pruning helps reduce unnecessary computation. The solution demonstrates how Backtracking can be applied to graph optimization problems.